POC 4 - Spectator

Appel d'offres ARES · TypeScript

Contexte

Un match ARES n'est pas enregistré comme une image de son état final, mais comme un journal d'événements : une liste ordonnée de faits, dans l'ordre où ils se sont produits (un robot s'est déplacé, un robot a tiré, un cristal a été ramassé). Le spectateur part de ce journal et reconstitue ce qui s'est passé.

Notions

Journal d'événements. Chaque ligne est un fait, numéroté par un ordre (`seq`). Exemple :

```
{ "seq": 0, "type": "MATCH_STARTED", "robots": ["A1", "B2"] }
{ "seq": 1, "type": "ROBOT_MOVED", "robot": "A1", "x": 14.2, "y": 48.0 }
{ "seq": 2, "type": "ROBOT_SHOT", "robot": "A1", "hit": "B2" }
{ "seq": 3, "type": "ROBOT_DAMAGED", "robot": "B2", "hp": 0, "alive": false }
```

Projection. Reconstituer l'état du match en appliquant les événements l'un après l'autre, à partir d'un état de départ. Chaque événement modifie un petit bout de l'état : `ROBOT_MOVED` change une position, `ROBOT_DAMAGED` change des points de vie.

Fourni

- Un court journal d'événements (une dizaine de lignes).

À faire

Écrire une fonction qui lit le journal et renvoie l'état final du match : positions des robots, points de vie, qui est encore en vie.

```
project(events: Event[]): MatchState

// exemple de valeur renvoyée :
{
  robots: {
    A1: { x: 14.2, y: 48.0, hp: 100, alive: true },
    B2: { x: 61.5, y: 33.1, hp: 0,  alive: false }
  }
}
```

Pas d'affichage, pas de graphisme. Juste le calcul de l'état.

Rendu

- Le code de la fonction.
- Un README court : les difficultés rencontrées.